

2.1 AWS Lambda: your first function

Nearly all of Encon's backend is Lambda functions. We don't run servers, so this is the unit of work you'll be writing for the rest of your time here.

NOTE: what is a Lambda handler???

A Lambda handler acts as the main entry point for an AWS Lambda function. When an event triggers your function, AWS Lambda runs this specific method or function to process the incoming data and execute your custom logic

Learning objectives

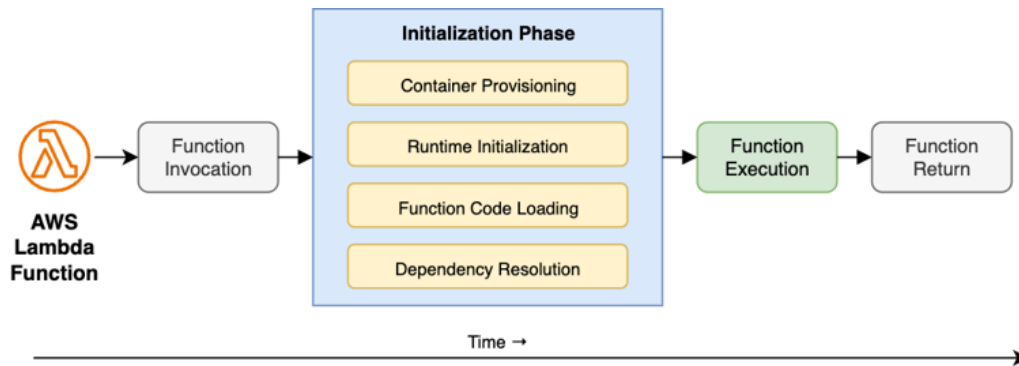
What does "serverless" actually mean, and what are you giving up in exchange?

'Serverless' is a cloud computing model where the cloud provider manages all backend infrastructure, servers, and scaling. For example, you don't have to create an EC2 machine, install Node.js on a server, keep the operating system updated, or manually scale the number of servers. AWS does it all for you.

What is a cold start, and when do you notice one?

Cold starts occur because serverless platforms like AWS Lambda are designed for cost-efficiency – you don't pay for compute resources when your code isn't running. As a result, Lambda only provisions resources when needed. A cold start happens when there isn't an existing execution environment available and a new one must be created. This can happen, for example, when a function is invoked for the first time after a period of inactivity or during a burst in traffic that triggers scale-up.

When AWS is initialising this new environment, there will be a bit of latency before the function runs.



If AWS can reuse an existing environment, you get a **warm start**, which is usually quicker. AWS says cold starts typically occur in under 1% of invocations, although they're more noticeable in infrequently invoked development/test functions.

What are the `{{event}}` and `{{context}}` arguments your handler receives?

event

The `event` is the **data sent into your Lambda**.

For my project, my event might be:

```
1 {  
2   "minTempC": 3,  
3   "maxTempC": 7,  
4   "fridgeId": "FRIDGE-001",  
5   "recordedBy": "Umar"  
6 }
```

Then my Lambda can access:

```
1 event.minTempC  
2 event.maxTempC  
3 event.fridgeId  
4 event.recordedBy
```

AWS converts the JSON test event into an object and passes it to your handler.

context

`context` contains information **about the Lambda invocation itself**, rather than your fridge reading.

For example, it can provide information such as the request ID and how much execution time remains.

Conceptually:

```
1 export const handler = async (event, context) => {
```

event = what was sent to me?

context = information about this Lambda execution

How does a Lambda get permission to do anything at all? (This is IAM, and it is the source of most "why doesn't this work" moments.)

Your Lambda does not automatically have permission to access every AWS service.

It gets permissions through an **IAM execution role**.

For example, later when you connect this function to DynamoDB, its role will need permission to perform the required DynamoDB actions.

That idea is important:

Lambda code says what the function wants to do. IAM determines whether AWS allows it to do it.

Task



• Do

- Create a Node.js Lambda in the AWS console in our development account. Name it with your own prefix, e.g. `{{yourname-coldchain-create-reading}}`
- Drop in the `{{getReadingStatus}}` function you wrote and tested in 1.7
- Write a test event that looks like a real reading — `{{minTempC}}`, `{{maxTempC}}`, `{{fridgeId}}`, `{{recordedBy}}` — and have the Lambda return the status it worked out
- Run it with an in-range reading and an out-of-range one, and check you get `{{ok}}` and `{{excursion}}`

• Nothing is being saved yet. That comes next ticket.

• Done when

- The function returns the right status for both a good and a bad test event
- You've noted in the worksheet where the *code* lives versus where the *configuration* lives (timeout, memory, role,

environment variables)

Understand Lambda → create Lambda → paste your business rule → create handler → deploy
→ create good test event → create bad test event → record results → document code vs configuration.

- ✓ understand Lambda
- ✓ create Lambda
- ✓ paste your business rule
- ✓ create handler
- ✓ deploy
- ✓ create good test event
- ✓ create bad test event
- ✓ document code vs configuration.

Understand Lambda

Previously we used Jest to test/execute the function. Now we want Lambda to sit around our function (Lambda is not replacing the function, it is providing a way for AWS to invoke it).

Creating a Lambda / Paste your business rule / Create handler

First we need to create the Lambda in the AWS console. (For this task in the project we are not using IaC; we're doing it manually in the console.) First I initialized the Lambda by going through the 'create' process, giving it a name, configurations, etc. Then I input the code below into the Lambda:

```
1 function getReadingStatus(minTempC, maxTempC) {
2   if (!Number.isFinite(minTempC) || !Number.isFinite(maxTempC)) {
3     throw new TypeError(
4       'minTempC and maxTempC must both be valid numbers'
5     );
6   }
7
8   if (minTempC < 2 || maxTempC > 8) {
9     return 'excursion';
10  }
11
12  return 'ok';
13 }
14
15 export const handler = async (event, context) => {
16   const status = getReadingStatus(
17     event.minTempC,
18     event.maxTempC
19   );
20
21   return {
```

```

22     fridgeId: event.fridgeId,
23     recordedBy: event.recordedBy,
24     minTempC: event.minTempC,
25     maxTempC: event.maxTempC,
26     status: status
27   };
28 };

```

Explanation of the code:

Notice how aspects of your original code are still in this new Lambda code. For example:

```

1  function getReadingStatus(minTempC, maxTempC)

```

Notice how there are new aspects as well:

```

1  export const handler = async (event, context) => {

```

This code is the entry point for the AWS Lambda. AWS documents the handler as the method Lambda invokes when an event arrives. (This works — can you see the event? Remember it's an event-driven process, so the events in the code are what the Lambda will be looking for to execute.)

Understanding what the handler is doing:

This:

```

1  event.minTempC

```

gets the minimum temperature from the event.

This:

```

1  event.maxTempC

```

gets the maximum.

They're passed into the tested function:

```

1  const status = getReadingStatus(
2    event.minTempC,
3    event.maxTempC
4  );

```

Then:

```

1  return {

```

sends the result back.

E.g. if the event is:

```

1  {
2    "minTempC": 3,
3    "maxTempC": 7,
4    "fridgeId": "FRIDGE-001",
5    "recordedBy": "Umar"

```

```
6 }
```

your function calculates:

```
1 | getReadingStatus(3, 7)
```

which returns:

```
1 | ok
```

and Lambda returns something like:

```
1 | {  
2 |   "fridgeId": "FRIDGE-001",  
3 |   "recordedBy": "Umar",  
4 |   "minTempC": 3,  
5 |   "maxTempC": 7,  
6 |   "status": "ok"  
7 | }
```

Deployment

Deployment is one thing that can potentially catch me out. You just need to remember that you need to click the deploy button when you edit code. It doesn't automatically change as you edit the code; you should deploy before testing.

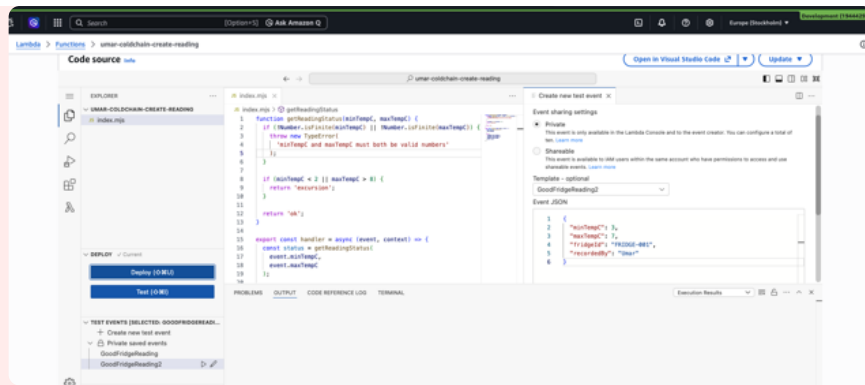
Create good test event

The first test I did was the "Good Test." It was successful because the function returned a result of "ok". The test was named GoodFridgeReading and the test is shown below:

```
1 | {  
2 |   "minTempC": 3,  
3 |   "maxTempC": 7,  
4 |   "fridgeId": "FRIDGE-001",  
5 |   "recordedBy": "Umar"  
6 | }
```

Error (problem that arose):

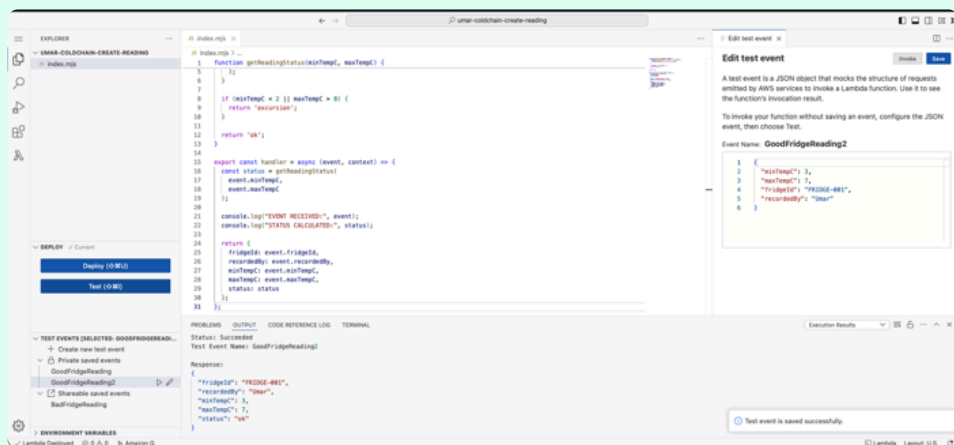
This test initially didn't work. I asked Hamza or Luke for help. I also tried using AI to change the test and create new tests, but nothing would output; you can see the screenshots below:



Nothing was outputting when I clicked test. I tried deploying and saving again but it still wouldn't run, so I wasn't sure what I had done wrong.

✓ Fix (problem solved):

All I needed to do was refresh the browser; it was a browser issue. Probably because the browser had been open since the day before, it was acting up, but now the tests are working. As you can see from the screenshots below:



Create bad test event

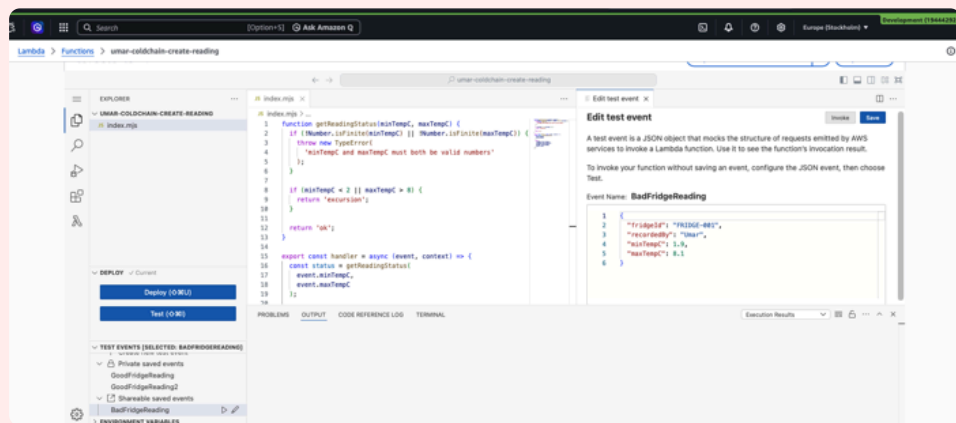
The second test was a bad test called 'BadFridgeReading'. The test is shown below:

```
1 {
2
3
4
5
6 }
```

```
{
  "minTempC": 1.9,
  "maxTempC": 8.1,
  "fridgeId": "FRIDGE-001",
  "recordedBy": "Umar"
}
```

✗ Error (problem that arose):

This test initially didn't work either; same thing as the good test—nothing happened.



✓ Fix (problem solved):

All I needed to do was refresh the browser; it was a browser issue. Probably because the browser had been open since the day before, it was acting up, but now the tests are working. As you can see from the screenshots below:



Document code vs configuration

This section specifically covers the question asked in the "You've noted in the worksheet where the code lives versus where the configuration lives." I was initially confused by the wording of this question; however, I have figured out that it means:

It is basically asking you to prove that you understand the difference between **the program you wrote** and **the AWS settings that control how that program runs**.

Think of your Lambda as two separate things

1. **Code = what your Lambda does**

This is the JavaScript you are currently writing in `index.mjs` , for example:

```
1 function getReadingStatus(minTempC, maxTempC) {
2   if (!Number.isFinite(minTempC) || !Number.isFinite(maxTempC)) {
3     throw new TypeError(
4       'minTempC and maxTempC must both be valid numbers'
5     );
6   }
7
8   if (minTempC < 2 || maxTempC > 8) {
9     return 'excursion';
10  }
11
12  return 'ok';
13 }
```

2. **Configuration = how AWS runs your Lambda**

Configuration is **not JavaScript**.

It is AWS settings surrounding your function.

For example:

Configuration	Simple meaning
Memory	How much RAM AWS gives your Lambda
Timeout	How long AWS allows the Lambda to run before stopping it
IAM Role	What AWS services the Lambda is allowed to access
Environment Variables	Values/settings you can give your code without hard-coding them

These live under the **Configuration** tab.

You can actually see that tab at the top above the AWS code:



So that's really what your ticket means by:

where the code lives versus where the configuration lives

It's asking:

"Can Umar identify which things are part of his JavaScript program and which things are AWS settings?"

Why does this separation matter?

Suppose later my Lambda needs to save readings into DynamoDB.

Your JavaScript might say:

```
1 | Save this reading into DynamoDB
```

But AWS could still respond:

```
1 | AccessDenied
```

Why?

Because the **IAM role** hasn't been configured to give the Lambda permission to access DynamoDB.

So:

```
1 | CODE
2 | "What should I do?"
3 |   ↓
4 | JavaScript
5 |
6 | CONFIGURATION
7 | "How am I allowed to run?"
8 |   ↓
9 | AWS settings
```

So the code shows what should happen; the configuration is how you would solve issues like no access, not enough assigned memory, etc.

Now to answer the question

You've noted in the worksheet where the *code* lives versus where the *configuration* lives?

In AWS Lambda, my JavaScript code is stored in the Code section of the function. This includes my `getReadingStatus` function and the Lambda handler. The Lambda configuration is stored separately under the Configuration section. This includes settings such as the function's

memory, timeout, IAM execution role, and environment variables. The code controls what the Lambda does, while the configuration controls how AWS runs the Lambda and what AWS resources it has permission to access.

 Questions / Queries / Pushback

- Ask Luke when he returns to explain what a context is; say you are confused because you thought the event was the data the Lambda looked for. Now that you've reached the section on context, it says context is information about the execution. I'm confused (show your definitions above).

Ask how they utilise Lambdas: how does the code you edit in VS Code get into the git repository and then to Lambdas? (I have an idea: it is to do with Terraform and IaC.)